Curso de IA para agentes

Fundación Cognitus

CURSO DE IA PARA AGENTES · MÓDULO 5

# Bot de Telegram para un spa

Taller paso a paso con OpenCode y Supabase

Objetivo del taller

```
$ solicitar -> registrar -> confirmar -> notificar
Citas persistentes y respuesta desde Telegram
# Incluye prompts, comandos, pruebas y publicación
```

| OPENCODE | TELEGRAM | SUPABASE |
| --- | --- | --- |

Guía de práctica. Precios y horarios ficticios. Revisión: 2 de octubre de 2026.

El proyecto inicial acompaña esta guía en taller-telegram-opencode.zip.

1. EL PROYECTO

# Qué vas a entregar

Vas a construir un recepcionista digital para un spa ficticio de manos y pies. La clienta consulta servicios, solicita una cita y recibe la decisión del responsable. Telegram será la interfaz, TypeScript ejecutará las reglas y Supabase conservará los datos y alojará el bot.

OpenCode escribirá el código y las pruebas. Tú copiarás los encargos, conectarás tus cuentas y comprobarás los resultados. No necesitas escribir TypeScript o SQL a mano. Sí debes revisar qué creó OpenCode y qué pruebas ejecutó.

| OPENCODE | TELEGRAM | SUPABASE |
| --- | --- | --- |
| Construye y verifica | Conversa y notifica | Guarda y ejecuta |

- Un bot publicado que funciona sin dejar tu computador encendido.
- Servicios, citas y estados guardados en Supabase.
- Confirmación, rechazo y cancelación con permisos por persona.
- Registro de avisos pendientes y fallidos.
- Evidencia de pruebas locales y una demostración en Telegram.

Trabajaremos con una sola estación y bloques de una hora. Los precios y duraciones son ficticios. La clienta puede tener una cita activa a la vez. El responsable confirma; solicitar todavía no equivale a tener una cita confirmada.

No construiremos pagos, recordatorios automáticos ni IA conversacional en esta práctica. La IA que ayuda a programar es OpenCode; el bot ejecuta reglas definidas.

# Qué necesitas

• OpenCode instalado y conectado a un proveedor de modelos.

• Node.js 20 o posterior, para ejecutar Supabase CLI mediante npx.

• Deno 2 o posterior, para verificar el TypeScript y ejecutar los scripts.

• Una cuenta en Telegram y una cuenta en Supabase.

• Una segunda cuenta de Telegram o un compañero para probar clienta y responsable por separado.

• El paquete inicial de este taller.

El acceso al modelo de OpenCode y los servicios en la nube pueden tener costos y límites según tus cuentas. El taller no presupone que todas las cuentas sean gratuitas.

Si ya tienes las herramientas, pasa al punto 3. Comprueba en tu terminal, una línea por vez:

Copiar y pegar

```
opencode --version
node --version
deno --version
```

Si falta Node.js, instala la versión LTS desde nodejs.org, reinicia la terminal y comprueba otra vez. Si falta OpenCode, sigue su guía oficial.

Para instalar Deno en macOS, Linux o Ubuntu de WSL:

Copiar y pegar

```
curl -fsSL https://deno.land/install.sh | sh
```

En Windows con PowerShell:

Copiar y pegar

```
irm https://deno.land/install.ps1 | iex
```

Cierra y vuelve a abrir la terminal. Comprueba `deno --version`. Si no se reconoce, usa las instrucciones de PATH que mostró el instalador. En WSL, las tres herramientas deben estar disponibles dentro de Ubuntu. Instalación de Deno.

# Abrir el paquete en OpenCode

1. Descomprime `taller-telegram-opencode.zip` en una carpeta de trabajo.
2. Conserva una copia del ZIP original para recuperación.
3. Abre la carpeta `proyecto-inicial` en OpenCode Desktop. Si usas terminal, entra a esa carpeta y ejecuta `opencode`.
4. Comprueba que ves `AGENTS.md`, `ESPECIFICACION.md`, `opencode.json`, `.env.example` y `scripts/`.

**No abras la carpeta de otro módulo ni construyas encima de un proyecto anterior.**

En OpenCode existen los agentes Plan y Build. Plan servirá para revisar; Build para crear archivos y ejecutar comprobaciones. En la interfaz de terminal puedes alternar con Tab; en Desktop usa el selector de agente. La configuración del paquete bloquea escrituras y comandos en Plan. Agentes de OpenCode.

Los bloques identificados como **PROMPT** se pegan en OpenCode. Los bloques identificados como **TERMINAL** se ejecutan en tu terminal, dentro de `proyecto-inicial`. No pegues los comandos de conexión en el chat de OpenCode.

4. OPENCODE · PLAN

# Pedir el plan

Selecciona **Plan** y pega este bloque completo.

**PROMPT 1**

PROMPT 1

```
Lee AGENTS.md, ESPECIFICACION.md, README.md y opencode.json. Todavía no crees ni cambies archivos ni ejecutes comandos.

Voy a construir el bot de Telegram del Spa Manos y Pies con TypeScript, Deno y Supabase Edge Functions. Soy principiante y seguiré un taller por etapas.

Explícame en español el recorrido de la clienta y el responsable. Propón un plan dividido en: base de datos, bot, permisos y recuperación de avisos, pruebas, publicación. Conserva exactamente el alcance de ESPECIFICACION.md.

Indica qué archivos crearás, cómo impedirás que dos personas ocupen el mismo horario y cómo conservarás la cita si Telegram falla. Distingue las pruebas con mocks de las que requieren una base real. No me pidas tokens, claves o contraseñas. No agregues servicios ni capacidades.
```

Puedes continuar cuando el plan incluya estas cinco cosas: horarios en Bogotá, cita pendiente antes de confirmar, validación del responsable, una restricción real en Postgres para ocupación y avisos persistentes.

Si falta alguna, pega:

Copiar y pegar

```
Ajusta el plan a ESPECIFICACION.md. Falta incluir uno o más de estos puntos: zona America/Bogota, confirmación humana, permisos por user_id, restricción de ocupación en Postgres y notificaciones persistentes. No construyas todavía.
```

# Crear la base del proyecto

Cambia a **Build** y pega:

**PROMPT 2**

PROMPT 2

```
Construye la etapa de estructura y base de datos del plan, dentro de esta carpeta.

1. Amplía deno.json con las tareas check y test para el proyecto completo y fija las
dependencias necesarias. Conserva las tres tareas telegram:identificar, telegram:conectar
y telegram:estado existentes. Crea supabase/config.toml con project_id = "spa-manos-pies"
y [functions.spa-telegram] verify_jwt = false.
2. Crea supabase/migrations/202610020001_spa.sql con las tablas, checks, índices, RLS,
privilegios y servicios ficticios de ESPECIFICACION.md. No uses DROP ni datos personales
reales.
3. Implementa spa_generar_horarios() y spa_mutar_cita. La mutación guarda el evento, la
cita y los avisos en una sola transacción; no repartas esos pasos en llamadas HTTP
separadas. Protege las RPC de anon/authenticated/PUBLIC.
4. Crea las RPC para reclamar y finalizar avisos con lease y token, según la
especificación. Limita los lotes y usa SKIP LOCKED.
5. Añade tests/integration.sql con BEGIN y ROLLBACK, aserciones reales y datos ficticios.
Incluye permisos, duplicados, ocupación, cancelación, vencimiento y avisos. Añade
instrucciones para comprobar concurrencia en dos sesiones separadas.
6. Crea evidencia/RESULTADOS.md: la integración SQL y Telegram real empiezan como
PENDIENTE.

Verifica la estructura y los scripts existentes sin leer .env. No crees archivos ficticios
solo para hacer pasar check. En esta etapa puede faltar todavía la función: registra qué
está pendiente para la etapa siguiente.

Al terminar muestra la ruta de la migración, las funciones creadas y las instrucciones
para ejecutar la migración una sola vez desde SQL Editor. No publiques ni conectes
cuentas.
```

**Comprueba:** existe el archivo SQL. OpenCode debe explicar que el índice único parcial protege el horario incluso con solicitudes simultáneas. Si solo propone “consultar si está libre y luego insertar”, pide corregirlo antes de continuar.

6. OPENCODE · BUILD

# Construir el bot

Sigue en **Build**.

**PROMPT 3**

PROMPT 3

```
Construye ahora el bot completo, usando las RPC y los archivos de la etapa anterior. Lee
otra vez ESPECIFICACION.md antes de editar.

Implementa la función spa-telegram y sus módulos: entrada, handler, repositorio y cliente
Telegram. Usa fetch para Telegram y supabase-js para la base; no agregues un framework ni
un modelo de IA.

Incluye /start, /help, Servicios, Agendar, Mi cita, resumen y Enviar solicitud. Incluye
confirmación y rechazo por el responsable, cancelación con segunda confirmación,
/pendientes paginado y /reintentar.

Verifica X-Telegram-Bot-Api-Secret-Token antes de leer JSON. Valida el evento, el chat
privado, los callbacks y los permisos en el servidor. ADMIN_USER_ID y ADMIN_CHAT_ID vienen
del entorno; nunca del mensaje. Usa fechas de Bogotá y revalida disponibilidad al crear la
solicitud.

Consulta o genera horarios de forma idempotente. No mantengas la conversación solamente en
memoria. No muestres citas de otras personas. Todos los precios y estados vienen de la
base.

Implementa los avisos persistentes y sus leases. No pierdas una cita por un error al
enviar. No prometas entrega o lectura: ok=true significa aceptado por Telegram. Si hay
timeout incierto, conserva revision; no reenvíes a ciegas.

Incluye validación de variables y compatibilidad con SUPABASE_SECRET_KEYS y
SUPABASE_SERVICE_ROLE_KEY inyectadas por Supabase. No copies claves de servidor a ejemplos
locales ni crees .env.

Ejecuta deno task check y corrige todos los errores. No publiques. Dime qué probarás a
continuación y qué faltaría si el código aún no está completo.
```

**Comprueba:** existe `supabase/functions/spa-telegram/index.ts`, y OpenCode reporta que `deno task check` terminó correctamente. Esta comprobación todavía no demuestra que Telegram esté conectado.

7. VERIFICACIÓN LOCAL

# Probar antes de conectar cuentas

### PROMPT 4

PROMPT 4

```
Completa las pruebas del bot con datos ficticios, reloj fijo y mocks. No leas .env ni
llames a servicios reales.

Prueba como mínimo: menú inicial; servicios; día y horario en Bogotá; fecha pasada;
horario ocupado; cita propia; cancelación ajena; confirmación por una persona que no es
responsable; callback manipulado; callback viejo; dos solicitudes de la misma persona;
replay de update_id; doble confirmación; aviso fallido después de guardar; HTTP 200 de
Telegram con ok=false; 429; timeout incierto; lease vencido; secreto incorrecto; JSON
inválido; mensajes de grupo y texto no reconocido.

Verifica que se muestra un error comprensible, que ninguna falla de envío borra la cita y
que los estados de aviso se guardan correctamente. Mantén scripts/telegram-config_test.ts
en la suite.

Ejecuta deno task check y deno task test. Corrige las fallas y repite solamente después de
corregir. Actualiza evidencia/RESULTADOS.md con lo que ejecutaste: no marques como
aprobadas la integración SQL, la concurrencia o Telegram real si no se ejecutaron.

Crea GUIA_PUBLICACION.md con los nombres exactos de las variables, ruta de la función y
comprobaciones posteriores. Sin secretos. Describe la limitación de recuperación: sin cron
los avisos avanzan con nuevos eventos o /reintentar, no por sí solos.
```

### TERMINAL

TERMINAL

```
deno task check
deno task test
```

No avances con pruebas fallidas. Si necesitas ayuda, copia a OpenCode únicamente el error sin credenciales:

Copiar y pegar

```
La comprobación falló. Aquí está el error sin credenciales: [PEGA EL ERROR]. Reproduce el
problema con datos ficticios, corrige su causa y ejecuta otra vez check y test. No
elimines la prueba ni reduzcas sus aserciones para hacerla pasar.
```

8. SUPABASE

# Crear el proyecto de Supabase

Esta parte la haces tú en el navegador.

1. Abre Supabase Dashboard.
2. Crea un proyecto nuevo llamado `spa-manos-pies` para esta práctica.
3. Guarda la contraseña de la base en tu gestor de contraseñas. OpenCode no la necesita.
4. Espera a que el proyecto esté disponible.
5. Copia el **Project ID o project ref** desde la configuración general. También aparece en la dirección del dashboard después de `/project/`.

No copies el nombre del proyecto en lugar del ID. El ID será una cadena parecida a `abcdefghijklmnopqrst`. No es una clave secreta.

# Ejecutar el SQL y revisar la base

1. Abre `supabase/migrations/202610020001_spa.sql` en tu editor.
2. Copia su contenido completo.
3. En Supabase entra a **SQL Editor** → **New query**.
4. Pega el SQL y pulsa **Run**. Ejecuta la migración una sola vez en este proyecto nuevo.
5. En **Table Editor**, comprueba que existen las seis tablas de la especificación.

Para comprobar datos, ejecuta en otra consulta:

SQL Editor

```
select id, nombre, precio_cop, duracion_min
from public.spa_servicios
order by id;

select id, inicio at time zone 'America/Bogota' as hora_bogota
from public.spa_horarios
order by inicio
limit 12;
```

Debes ver tres servicios y horarios futuros. Los horarios no deben aparecer como 14:00 cuando esperabas 09:00: usa la columna `hora_bogota` de esta consulta.

Ahora abre `tests/integration.sql`, copia el contenido y ejecútalo en SQL Editor. Debe terminar sin errores y con ROLLBACK. No deja las citas ficticias de prueba. Si falla, entrega el error sin credenciales a OpenCode; que corrija el SQL y prepare una migración incremental. **No vuelvas a ejecutar toda la migración inicial ni borres la base para ocultar el problema.**

Marca integración como aprobada solo después de observar ese resultado. La prueba de dos sesiones se realiza aparte siguiendo las instrucciones generadas; si no se hace, conserva concurrencia como pendiente.

# Crear el bot de Telegram

1. Abre BotFather oficial.
2. Envía `/newbot`.
3. Escribe el nombre visible, por ejemplo `Spa Manos y Pies Taller`.
4. Elige un username único que termine en `bot`, por ejemplo `spa_manos_pies_tu_nombre_bot`.
5. Conserva el token que entrega BotFather. No lo pegues en OpenCode ni lo incluyas en la evidencia.
6. Abre el enlace del nuevo bot, pulsa **Iniciar** y envía `/identificar` desde la cuenta que será responsable.

Todavía no responderá. Eso es normal: aún no conectamos la función.

# Preparar secretos y obtener los IDs

En tu terminal, dentro del proyecto, crea la copia local.

**TERMINAL macOS, Linux o WSL**

TERMINAL macOS, Linux o WSL

```
cp .env.example .env
```

**TERMINAL PowerShell**

TERMINAL PowerShell

```
Copy-Item .env.example .env
```

Abre `.env` tú mismo en un editor de texto. Reemplaza `REEMPLAZAR` en TELEGRAM_BOT_TOKEN por el token de BotFather. No incluyas espacios alrededor de `=`.

Genera el secreto del webhook:

**TERMINAL**

TERMINAL

```
deno eval "console.log(crypto.randomUUID().replaceAll('-', ''))"
```

Copia el valor generado a TELEGRAM_WEBHOOK_SECRET. Es una clave distinta del token de BotFather. Guarda exactamente la misma para Supabase en el paso siguiente.

En TELEGRAM_WEBHOOK_URL reemplaza `REEMPLAZAR` por el Project ID:

Copiar y pegar

```
https://TU_PROJECT_ID.supabase.co/functions/v1/spa-telegram
```

Guarda el archivo. Ejecuta:

**TERMINAL**

TERMINAL

```
deno task telegram:identificar
```

El script muestra las personas que enviaron `/start` o `/identificar` al bot. Elige la fila del responsable y conserva ADMIN_USER_ID y ADMIN_CHAT_ID. En chats privados suelen coincidir, pero cumplen funciones diferentes.

Si no aparecen personas, vuelve a enviar `/identificar` al bot correcto y repite el comando. Este paso se hace **antes de registrar el webhook**. Usa un bot nuevo del taller; el script no elimina conexiones existentes.

# Guardar las variables en Supabase

En el dashboard abre **Edge Functions** → **Secrets**. Crea estas cuatro variables:

| Nombre exacto | Valor |
|---|---|
| TELEGRAM_BOT_TOKEN | Token de BotFather |
| TELEGRAM_WEBHOOK_SECRET | Secreto generado en el paso anterior |
| ADMIN_USER_ID | ID del responsable obtenido por el script |
| ADMIN_CHAT_ID | ID de su chat privado obtenido por el script |

Guarda los valores. No publiques capturas de este formulario. SUPABASE_URL y las claves de servidor las inyecta Supabase en la función alojada; no debes copiar ninguna clave de Supabase a `.env`. Variables de Edge Functions.

13. PUBLICACIÓN

# Publicar la función

Publicar subirá el código del bot al proyecto nuevo de práctica. Hazlo después de que check, test e integración SQL hayan pasado.

Desde tu terminal, dentro de `proyecto-inicial`:

**TERMINAL**

TERMINAL

```
npx --yes supabase@latest login
```

Completa la autenticación en el navegador. Si pide un token de acceso, introdúcelo únicamente en el flujo de login; no lo entregues a OpenCode.

Reemplaza `TU_PROJECT_ID` y ejecuta:

Copiar y pegar

```
npx --yes supabase@latest functions deploy spa-telegram --project-ref TU_PROJECT_ID --use-api
```

No ejecutes `supabase init` sobre esta carpeta: OpenCode ya creó la configuración. `--use-api` solicita el empaquetado remoto de la CLI; no necesitas arrancar una base local ni Docker.

En Supabase abre `spa-telegram` y comprueba:

• La función está publicada.

• La URL termina en `/functions/v1/spa-telegram`.

• **Verify JWT** está desactivado para esta función, como indica `verify_jwt = false` en config.toml.

Telegram no envía un JWT de Supabase. La autenticación de este webhook está implementada en el código con TELEGRAM_WEBHOOK_SECRET. No desactives la comprobación del secreto. Seguridad de Edge Functions.

14. WEBHOOK

# Conectar Telegram a la función

Revisa que `.env` contenga la URL correcta y el mismo secreto que guardaste en Supabase.

El siguiente comando registra el webhook y descarta los mensajes de prueba acumulados antes de conectarlo. Por eso usamos un bot nuevo para el taller.

**TERMINAL**

TERMINAL

```
deno task telegram:conectar
```

Debes ver `"configurado": true`. Esto confirma que Telegram aceptó la configuración; todavía debes comprobar que el bot responde.

Comprueba la conexión:

Copiar y pegar

```
deno task telegram:estado
```

Debes ver `"conectado_a_esta_funcion": true`. Abre el bot y envía `/start`. Debe aparecer el menú.

El script transmite el token directamente a Telegram y no lo imprime. Nunca construyas URLs con el token en el navegador ni las pegues en el chat. Configuración de webhooks de Telegram.

# Probar el recorrido completo

Usa dos cuentas: A es el responsable, B es la clienta. Ambas deben haber iniciado el bot. Los bots necesitan que la persona abra la interacción antes de poder escribirle por privado. Funciones de los bots.

| Prueba | Qué haces | Qué debes observar |
|---|---|---|
| Servicios | B pulsa Servicios | Tres servicios y sus precios ficticios |
| Solicitud | B selecciona servicio, fecha, hora y envía | Número de cita; estado pendiente; aviso a A |
| Doble clic | B pulsa otra vez el botón de enviar anterior | Una sola cita activa, sin duplicar la solicitud |
| Confirmación | A confirma | B recibe confirmación; base queda confirmada |
| Doble confirmación | A pulsa nuevamente el botón antiguo | No se crea una segunda transición |
| Consulta | B pulsa Mi cita | Solo aparece su cita |
| Cancelación | B confirma cancelar | Estado cancelada y horario liberado |
| Rechazo | B solicita otra cita y A rechaza | Estado rechazada y opción de solicitar de nuevo |
| Permisos | B escribe /pendientes y /reintentar | Acceso denegado sin exponer citas |
| Texto libre | B escribe “hola, qué puedes hacer” | Ayuda con opciones |

Para probar el horario ocupado, usa una tercera cuenta C o una segunda clienta del grupo. Dos personas eligen el mismo horario antes de enviarlo. Solo una obtiene la cita; la otra recibe el aviso de ocupación. Si no tienes tercera cuenta, conserva esa prueba como pendiente y verifica al menos su equivalente en SQL.

Comprueba los estados en SQL Editor:

SQL Editor

```
select id, servicio_id, horario_id, estado
from public.spa_citas
order by id desc
limit 10;

select id, estado, intentos, error_codigo, telegram_message_id
from public.spa_notificaciones
order by id desc
limit 20;
```

Que una notificación quede `enviada` significa que Telegram aceptó el mensaje. No demuestra que la clienta lo leyó.

# Probar una falla de notificación

Solo con las cuentas de práctica:

1. Asegúrate de que B no tenga cita activa.
2. B solicita una cita y espera el aviso de solicitud registrada.
3. B bloquea el bot desde Telegram.
4. A confirma la cita desde su aviso o `/pendientes`.
5. Revisa la base: la cita debe seguir `confirmada`; el aviso a B debe quedar `fallida` por el rechazo de Telegram.
6. B desbloquea el bot y envía `/start`.
7. B consulta Mi cita y debe ver que está confirmada.

El bot no reintenta automáticamente errores permanentes ni estados `revision`. `/reintentar` procesa solamente avisos elegibles; no resuelve un bloqueo de la clienta.

Si un aviso quedó pendiente por un problema temporal, A puede escribir `/reintentar` cuando haya pasado la espera prevista. En esta práctica no hay cron: sin un evento nuevo, la cola no avanza por sí sola. Un timeout incierto requiere revisar el caso; reenviar puede duplicar un mensaje aunque la cita nunca se duplique.

17. REVISIÓN FINAL

# Pedir la revisión final a OpenCode

Entrega solo resultados sin secretos. Reemplaza el texto entre corchetes.

**PROMPT 5**

PROMPT 5

```
Revisa el proyecto terminado contra ESPECIFICACION.md. Ejecuta check y test locales. No
leas .env ni invoques servicios reales.

Estos son los resultados que observé manualmente:
- Integración SQL: [APROBADA o ERROR observado].
- Concurrencia en dos sesiones: [APROBADA o NO EJECUTADA].
- Menú y servicios en Telegram: [resultado].
- Solicitar, confirmar, consultar, cancelar y rechazar: [resultado].
- Doble clic y permisos: [resultado].
- Horario ocupado con dos clientas: [resultado o NO EJECUTADA].
- Bloquear bot y confirmar: [estado de la cita y del aviso].

Actualiza evidencia/RESULTADOS.md distinguiendo lo que ejecutaste tú de lo que observé yo.
No inventes pruebas pendientes ni incluyas información personal. Si hay un defecto,
corrige el código y dime si debo repetir SQL, publicar otra vez o probar Telegram.

Entrega una lista breve con requisitos cumplidos, pendientes y limitaciones. No agregues
nuevas funciones ni declares el bot listo para operación comercial.
```

Si OpenCode corrigió la función, repite el comando de publicación del paso 13 y vuelve a probar el flujo afectado. Si mantienes la misma URL y secreto, no hace falta registrar otra vez el webhook.

18. ENTREGA

# Evidencia de entrega

Entrega la carpeta del proyecto o su repositorio, **sin** `.env` **ni secretos**, junto con:

• Username público del bot.

• Una captura del menú.

• Una captura de solicitud y confirmación con datos personales ocultos.

• Una captura o consulta de estados de la base, sin credenciales.

• `evidencia/RESULTADOS.md` con las pruebas realizadas y pendientes.

Antes de comprimir, excluye archivos privados. No uses “comprimir toda la carpeta” sin revisar su contenido. `.gitignore` evita que Git incluya `.env`, pero no lo excluye automáticamente de un ZIP.

# Resolver problemas frecuentes

| Problema | Qué revisar |
|---|---|
| Deno o Node no se reconoce | Reinicia la terminal; comprueba instalación y PATH en la misma ruta de trabajo |
| OpenCode pide permiso para cada comando | La configuración permite check/test, pero pide revisar otros comandos; no autorices lecturas de secretos |
| El script dice que falta una variable | Edita .env tú mismo; no pegues su contenido en OpenCode |
| identificar no muestra personas | Envía /identificar en el bot correcto antes de registrar webhook |
| identificar dice que ya existe webhook | No lo borres a ciegas; usa tus IDs guardados o un bot nuevo de taller |
| Telegram muestra 401 | Si procede de la API de Telegram, revisa token; si procede de Supabase, revisa Verify JWT y secreto |
| La función responde 500 o 503 | Mira Logs; revisa variables, migración y nombres de RPC; comparte solo el error sin secretos |
| El webhook apunta al proyecto equivocado | Corrige TELEGRAM_WEBHOOK_URL y registra nuevamente |
| La clienta no recibe confirmación | Comprueba que inició y no bloqueó el bot; revisa spa_notificaciones |
| Aparece “horario ocupado” tras ver disponibilidad | Otra solicitud pudo ocuparlo; la validación final debe ofrecer elegir otro |
| No hay horarios | Ejecuta `select public.spa_generar_horarios();` y revisa fechas Bogotá |
| “relation already exists” | Se repitió una migración; pide a OpenCode una corrección incremental sin borrar datos |
| /reintentar no envía un aviso | Revisa espera, máximo de intentos y estado; fallida/revision no son reintentos automáticos |

Para recuperar el proyecto inicial, descomprime el ZIP original en otra carpeta. Conserva el proyecto que falló para revisar sus errores; no pierdas las credenciales privadas que guardaste por separado.

DOCUMENTACIÓN

# Fuentes oficiales

Las pantallas y comandos se contrastaron con documentación oficial el 2 de octubre de 2026. Si una interfaz cambia, identifica la función equivalente y conserva las comprobaciones.

- Reglas de proyecto de OpenCode.
- Permisos de OpenCode.
- Publicación de Supabase Edge Functions.
- Creación de funciones desde el dashboard.
- API oficial de Telegram.